SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**V.Madhulika**
Name	**192312620**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

<u>Descriptive Test</u>

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I ***<u>V.Madhulika(192312620)</u>*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

V.Madhulika

Signature of the student:

<u>RUBRICS FOR CALCULATION:</u>

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Question 1: Types of Intelligent Agents in a Smart Home Automation System

A smart home system must manage lighting, temperature, and security using real-time inputs (motion, temperature, time of day) while also learning user preferences over time. Each classical agent type handles this task differently.

1. Simple Reflex Agent :

A simple reflex agent acts only on the current percept using fixed condition-action rules, with no memory of the past and no model of the world.

- In this system: "IF motion detected AND room is dark → turn on light"; "IF temperature > 28°C → turn on AC"; "IF time = 10 PM → lock doors."
- Strength: extremely fast, simple, and reliable for immediate safety-critical reactions.
- Weakness: cannot distinguish context - e.g., it cannot tell whether a room is briefly unoccupied or someone merely stopped moving, so it may switch lights off/on erratically and cannot optimise energy use or learn anything.

2. Model-Based Reflex Agent :

This agent maintains an internal model/state of the world to handle information that is not currently visible to its sensors.

- In this system: it keeps track of which rooms are likely occupied even during brief gaps in motion detection, remembers recent temperature trends, and infers whether the house is empty based on a history of sensor events rather than a single reading.
- Strength: handles partial observability - e.g., avoids flickering lights off the instant a person is briefly still.
- Weakness: still reacts based on rules over its internal state; it has no explicit notion of a target it is trying to reach, so it cannot plan ahead.

3. Goal-Based Agent:

A goal-based agent has explicit goals and chooses actions (or sequences of actions) that are expected to achieve them, reasoning about the future rather than just the present state.

- In this system: goals such as "maintain occupant comfort," "ensure the house is secure when unoccupied," or "pre-cool the living room by 6 PM before the occupant normally arrives" drive its behaviour.
- Strength: supports planning ahead (e.g., pre-heating a room based on the user's known schedule) rather than only reacting to the present moment.
- Weakness: if multiple goals conflict (e.g., "maximise comfort" vs. "minimise energy use"), a purely goal-based agent has no mechanism to decide which goal should win, or by how much.

4. Utility-Based Agent :

A utility-based agent assigns a numeric utility (desirability) value to each possible outcome and chooses the action that maximises expected utility, allowing it to make trade-offs between competing objectives.

- In this system: it can weigh comfort (maintaining 22°C), energy cost (avoiding unnecessary AC/heating runtime), and security priority together in a single utility function, rather than treating each as a rigid on/off goal.

- Strength: naturally resolves conflicts between comfort, cost, and security by picking the action with the best overall trade-off, not just the first goal satisfied.
- Weakness: requires a well-designed utility function; if the weights given to comfort, energy, and security are wrong, the agent's decisions will be systematically biased.

Most Effective Design: A Hybrid, Learning, Utility-Based Agent

Since the smart home must react instantly to safety-critical events, track state that is not always directly observable, pursue multiple explicit objectives, and balance trade-offs between comfort/energy/security - while also improving over time from the user's behaviour - no single classical agent type is sufficient on its own. The most effective design is a hybrid architecture:

- A model-based core continuously tracks occupancy, temperature history, and security status even when sensors briefly report nothing.
- A thin simple-reflex layer on top of it handles safety-critical, must-react-instantly actions (e.g., emergency lighting, intrusion alarms) without waiting for deliberation.
- Goal-based reasoning defines the explicit objectives the system pursues (comfort, security, scheduled pre-conditioning).
- A utility-based evaluation layer resolves conflicts between these goals (e.g., trading a little comfort for a significant energy saving) by scoring candidate actions.
- A learning component (as in a general learning agent: performance element + critic + learning element) continually adjusts the model and the utility function based on observed user preferences, so the system gets better at anticipating comfort needs over time.

This layered hybrid - a learning, utility-based agent built on a model-based/goal-based foundation, with a reflex layer for urgent safety responses - is what allows the system to be simultaneously fast, context-aware, personalised, and optimal across comfort, energy efficiency, and security.

Question 2: Robot Navigating an Unknown Maze

A robot placed in an unknown maze, with no prior knowledge of the layout, must find a path to the exit. Uninformed search strategies - which use no domain knowledge beyond the problem definition itself - are the natural starting point since no heuristic information about the maze is available.

How UCS Helps ?

Uniform Cost Search (UCS) expands the node with the lowest cumulative path cost g(n) so far, using a priority queue rather than a simple FIFO queue. As the robot moves through the maze, each newly discovered cell is added to the frontier along with its accumulated cost, and the robot always proceeds toward the cheapest unexplored option.

- Useful when moves are not all equally costly - e.g., some maze cells may represent rough terrain, inclines, or slower corridors that cost more to traverse than open floor.
- Guarantees the least-cost path to the exit is found, provided all step costs are positive.

How IDS Helps ?

Iterative Deepening Search (IDS) repeatedly performs a depth-limited depth-first search, increasing the depth limit by one each round (limit = 0, then 1, then 2, ...) until the exit is found.

- Particularly suited to an unknown maze because the robot has no idea in advance how far away the exit is (the search depth is unknown), and IDS does not need that information ahead of time.
- Behaves like DFS in terms of memory use (only needs to remember the current path), while still being complete and, for equal step costs, optimal - like BFS.

Feature	*Uniform Cost Search (UCS)*	*Iterative Deepening Search (IDS)*
Search Strategy	*Lowest path cost first*	*Increasing depth limits*
Complete	*Yes*	*Yes*
Optimal	*Yes (positive costs)*	*Yes (equal step costs)*
Time Complexity	O(b1+⌊C*/ε⌋)	*(O(b^d))*
Space Complexity	*Very High*	*Low*
Best Used When	*Costs vary*	*Memory is limited*

Relating the Problem to Different Types of Intelligent Agents :

- Simple reflex agent: could make the robot move immediately (e.g., "turn away from a wall on contact"), but with no memory it can loop endlessly in a maze and never systematically reach the exit - unsuitable alone.
- Model-based reflex agent: essential here, since the maze is unknown and only partially observable through the robot's local sensors; it must build and update an internal map of visited cells and known walls to avoid revisiting dead ends.
- Goal-based agent: defines the exit as an explicit goal state and applies a search algorithm (UCS or IDS) over its internal map to plan a route toward it, rather than merely reacting to immediate surroundings.
- Utility-based agent: relevant if the maze has cells with different traversal costs (e.g., rough terrain, water, risk of damage); it can select the path that best balances total cost, safety, and time rather than just the fewest steps.

How agent design affects decision-making: a purely reflex-based robot cannot systematically explore or backtrack in a maze, so at minimum a model-based agent is required to remember what has already been explored. Layering goal-based reasoning on top lets the robot direct its search toward the known/inferred exit rather than wandering randomly, and adding a utility-based layer allows it to make sensible trade-offs when some routes are safer or cheaper than others. The choice of search algorithm then determines how that goal-directed planning is actually carried out.

Recommended Combination and Justification :

The most effective design is a model-based, goal-based agent (with a utility-based layer where costs vary) that defaults to Iterative Deepening Search, escalating to Uniform Cost Search when the maze has cells with unequal traversal costs.

- Model-based + goal-based is necessary at minimum, because the maze is unknown and the robot must both remember what it has explored and reason explicitly about reaching the exit.
- IDS is the better default uninformed strategy here because a real robot has limited onboard memory, and the depth of the maze (distance to the exit) is not known in

advance - IDS handles both constraints gracefully while remaining complete and optimal for equal-cost moves.

- UCS becomes the better choice specifically when the maze's cells have unequal traversal costs (rough terrain, inclines, hazards), since only UCS guarantees the least-cost path in that situation - which is where a utility-based evaluation of routes also becomes meaningful.